

第5章 内存管理

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 GC 语言的内存模型:JS 里"变量在栈、对象在堆",但程序员看不到栈上指针
- 掌握 V8 的垃圾回收机制:分代、标记-清除、增量标记,以及"GC 不可控"的含义
- 掌握内存泄漏四大场景:全局变量、闭包、定时器、事件监听,以及排查工具
- 理解 WeakMap/WeakRef 的弱引用语义与生产用途
- 理解 Buffer 与 ArrayBuffer:二进制数据的两种容器与 C 的字节数组对照
- 掌握资源释放惯用法:try/finally 与连接/句柄的关闭
- 理解"C 中怎么做→JS 中怎么做":从手动 malloc/free 到自动回收

💡 从 C 到 JavaScript:本章主题如何迁移

C 程序员与内存的关系是"亲力亲为":malloc 申请、free 释放、野指针、双重释放、内存泄漏、悬垂指针,每一个都是深夜事故的常客。JS 把这个负担整体收走:垃圾回收器(GC)自动管理对象生命周期,没有 malloc/free,没有野指针,数组越界也只是 undefined 而不是写坏内存。

但"自动"不等于"不会泄漏"。GC 只能回收"不再被引用的对象";只要还有一条引用链活着,对象就永远留在堆里。C 的泄漏是"忘了 free",JS 的泄漏是"忘了断引用"。所以本章的核心不是"怎么释放",而是理解引用图:谁还握着这个对象。

V8(Node 与 Chrome 的引擎)把堆分成新生代与老生代,新生代用复制式 Scavenge 快速回收短命对象,老生代用标记-清除 + 增量标记清理长寿对象。GC 是自动的、不确定的:你不能手动触发"完整回收"(老代码的 `global.gc()` 只在 `--expose-gc` 下可用且仅用于调试),只能观察与调优。

另一个重要的迁移:JS 没有"栈上结构体"的概念,一切对象都在堆上;变量只是引用。这解释了为什么"把一个对象传给函数"永远是共享的(第4章),也解释了为什么深拷贝需要专门手段。二进制数据则是本章的新知识点:Buffer(Node)与 ArrayBuffer(标准),它们与 C 的 `unsigned char buf[N]` 直接对应。

📦 前置知识自查

- C 的栈/堆/静态区内存布局,malloc/free/calloc/realloc 的生命周期管理
- C 的指针、悬垂指针与内存泄漏概念
- C 的局部变量生命周期(函数返回即销毁)
- 静态变量 static 的持久性
- C 的结构体拷贝与字节数组 `unsigned char[]`

🚀 本章学习路线

1. **第一阶段(模型)**:读 §5.1–§5.2,建立"引用图 + 可达性"心智模型
2. **第二阶段(泄漏)**:读 §5.3–§5.4,亲手复现四大泄漏场景,学会用 `--heap-prof` 观察
3. **第三阶段(二进制)**:读 §5.5–§5.6,练习 Buffer 与 ArrayBuffer 的基本操作
4. **第四阶段(实验)**:完成章末"泄漏复现与修复 + 内存观察"
5. **验收标准**:能独立说出四种泄漏场景与各自修复,能画出任意两对象的引用关系

本章知识导图

- 第5章 内存管理
 - §5.1 内存模型:栈/堆、引用与可达性
 - §5.2 V8 回收机制:分代、标记-清除、不可控性
 - §5.3 内存泄漏四大场景与排查工具
 - §5.4 WeakMap / WeakRef 弱引用
 - §5.5 Buffer 与 ArrayBuffer 二进制数据
 - §5.6 资源释放惯用法:finally 与句柄关闭
 - §5.7 生产实践:泄漏检查清单、性能建议

§5.1 内存模型:从 malloc/free 到引用图

对比项	C	JS
对象位置	栈(局部)或堆(malloc)	对象都在堆;变量只是堆上的引用
释放	free() 手动释放	GC 自动,不可手动
悬垂指针	存在(use-after-free)	不存在(GC 保证对象存活到无人引用)
泄漏	忘了 free	忘了断引用
双重释放	UB,崩溃	不可能
数组越界	写坏相邻内存(UB)	返回 undefined,不损坏内存
查看占用	sizeof / valgrind	process.memoryUsage() / heap snapshot

```
// 引用图的直觉:变量指向对象
let a = { data: '大对象' }; // 对象在堆上,a 是引用
let b = a; // b 也指向同一个对象
a = null; // 断掉一条引用
// 对象仍被 b 引用,不会回收
console.log(b.data); // 大对象
b = null; // 现在没有任何引用,GC 随时回收

// 数组/对象嵌套:整棵引用树都活着
const root = { list: [{ id: 1 }, { id: 2 }] };
root.list = null; // 断掉子树根,两个对象变成垃圾
// 但 root 本身还活着

// 查看进程内存(Node)
console.log(process.memoryUsage());
// { rss: ..., heapTotal: ..., heapUsed: ..., external: ..., arrayBuffers: ... }
```

★ 重点结论:GC 回收的条件 = 不可达

一个对象能被回收,当且仅当 从根(全局对象、当前执行栈)出发无法到达它。环状引用(对象互相指)在 JS 里 不是 泄漏——C++ 的 shared_ptr 循环引用要手动破除,JS 的 GC 从根出发标记,互相指但没有根的环一样会被回收。这消除了 C++ 里最头疼的一类问题。

§5.2 V8 的垃圾回收:分代与标记-清除

V8 将堆分成两个代:

- **新生代(young generation)**:放刚创建的对象,容量小,用 **Scavenge(复制算法)** 回收——把存活对象复制到另一半空间,其余整体清空。大多数对象活不过几次回收。
- **老年代(old generation)**:存活多次回收的对象晋升到这里,用**标记-清除(Mark-Sweep)**与**增量标记**:先标记所有可达对象,再清除不可达对象;增量标记把大停顿拆成小片,避免卡顿。

```
// GC 不可控性的演示:创建大量临时对象后,内存并不立即回落
function createGarbage() {
  const tmp = [];
  for (let i = 0; i < 1e5; i++) {
    tmp.push({ i, s: '临时数据'.repeat(10) }); // 1e5 个对象
  }
  return tmp.length; // 返回后 tmp 变成垃圾
}
const before = process.memoryUsage().heapUsed;
console.log('创建垃圾:', createGarbage());
global.gc?(); // Node 加 --expose-gc 时可用,否则 undefined,不报错
const after = process.memoryUsage().heapUsed;
console.log('堆增量约:', Math.round((after - before) / 1024), 'KB');

// 生产调优:启动参数控制堆上限
// node --max-old-space-size=4096 app.js # 老年代上限 4GB
// node --expose-gc app.js # 暴露 global.gc()(仅调试)
```

⚠ 常见误区 5.1:以为"GC 会立刻回收"

GC 的时机由引擎决定:内存压力大、代际晋升、空闲时都可能触发。你无法保证"置空引用后内存马上回落"。性能测试里"创建 → 置空 → 立刻看内存"得到的数据不可靠;要看趋势,用 `--heap-prof` 或 Chrome DevTools 的 Memory 面板拍堆快照,对比前后引用图。

§5.3 内存泄漏四大场景

泄漏 = 对象长期被引用且不再需要。四大高频场景:

5.3.1 场景一:全局变量/全局容器无限增长

```
// ❌ 泄漏:全局数组无限追加
const cache = []; // 模块级 = 全局生命周期
function logEvent(e) {
  cache.push({ at: Date.now(), e }); // 永远有人引用,永不回收
}
for (let i = 0; i < 100; i++) logEvent('点击');

// ✅ 修复:限制大小(固定上限的环形缓冲)
const bounded = [];
function logEventSafe(e) {
  bounded.push({ at: Date.now(), e });
  if (bounded.length > 100) bounded.shift(); // 超限丢弃最旧
}
console.log(bounded.length); // 100,封顶

// 全局变量另一个形态:意外挂到全局
// ❌ 严格模式下 this 是 undefined,不会误挂全局;但忘写声明的变量在非严格模式会变全局
'use strict';
function bad() {
  // leak = 1; // 严格模式下 ReferenceError;非严格模式会创建全局变量!
}
```

5.3.2 场景二:闭包意外持有大对象

```
// ❌ 泄漏:闭包捕获了大对象,即使只需要小字段
function heavy() {
  const bigData = new Array(1e6).fill('x'); // 800KB 级别
  const id = 'user-1';
  return () => id; // 只需要 id,但闭包持有整个环境
}

const fn = heavy(); // bigData 一直被 fn 闭包引用

// ✅ 修复:让闭包只捕获需要的东西(把大对象放在另一作用域)
function light() {
  const id = 'user-1';
  {
    const bigData = new Array(1e6).fill('x'); // 块级作用域,函数返回后不可达
    console.log(bigData.length); // 只在块内使用
  }
  return () => id; // 闭包只捕获 id
}

const fn2 = light();
console.log(fn2());
```

5.3.3 场景三:定时器/轮询未清理

```
// ❌ 泄漏:setInterval 永不清理,回调及其闭包永不释放
const data = [];
const timer = setInterval(() => {
  data.push(Date.now()); // data 无限增长
}, 100);

// ✅ 修复:明确停止条件与清理
const timer2 = setInterval(() => {
  if (data.length > 10) {
    clearInterval(timer2); // 条件满足就停
    console.log('定时器已清理,元素数:', data.length);
  } else {
    data.push(Date.now());
  }
}, 100);

// Node 进程不退出检查:进程是否还挂着 handle
setTimeout(() => {
  console.log('Node 主动退出(即使有未清定时器,但生产要清干净)');
  clearInterval(timer);
  process.exit(0);
}, 500);
```

5.3.4 场景四:事件监听器重复注册

```
const { EventEmitter } = require('node:events');

// ❌ 泄漏:每调用一次就注册一个新监听,旧的永远在
function subscribeOnce(emitter) {
  emitter.on('data', (chunk) => {
    console.log('收到:', chunk.length);
  });
}

const em = new EventEmitter();
for (let i = 0; i < 1000; i++) subscribeOnce(em); // 1000 个监听!
console.log('监听器数量:', em.listenerCount('data')); // 1000

// ✅ 修复:命名函数 + off 清理,或 once
function handler(chunk) { console.log('收到:', chunk.length); }
em.on('data', handler);
em.off('data', handler); // 用完即移除
em.once('data', (c) => console.log('一次性:', c.length)); // 只触发一次自动移除
em.emit('data', Buffer.from('hello'));
console.log('剩余监听器:', em.listenerCount('data')); // 0
```

工程建议 5.1:泄漏排查三板斧

① `node --inspect app.js` + Chrome DevTools Memory 面板:录制堆快照,对比前后,按"Retained Size"排序找大对象;② 连续采样两次快照的 Delta 视图,看哪些构造函数实例数只增不减(典型的监听器/闭包泄漏);③ `process.memoryUsage()` 打点监控,配合 `--max-old-space-size` 让泄漏快速触发 OOM 报警而不是拖垮服务器。Node 20 还支持 `node --heap-prof` 直接产出 `.heapprofile` 文件离线分析。

§5.4 WeakMap 与 WeakRef:弱引用

WeakMap 的键不阻止垃圾回收:键对象变成垃圾时,键值对自动消失。典型用途:给对象附加元数据(缓存、私有数据),而不延长对象生命周期。WeakSet 同理。注意:WeakMap 的键必须是对象,不可遍历,没有 size。

```

// 缓存元数据:WeakMap 不会让键对象"永远活着"
const meta = new WeakMap();

function track(obj) {
  meta.set(obj, { visits: (meta.get(obj)?.visits ?? 0) + 1 });
}

let user = { name: '李雷' };
track(user);
track(user);
console.log(meta.get(user)); // { visits: 2 }
user = null; // 唯一引用断开
// meta 中的条目随之消失(GC 时),不会造成泄漏

// 对照:Map 会让键对象永远活着 ← 这就是"Map 版缓存"泄漏的根源
const strong = new Map();
strong.set(user, '某数据');

// 典型生产用途:缓存计算结果,键是"不可控的外部对象"
const cache = new WeakMap();
function compute(key) {
  if (!cache.has(key)) {
    cache.set(key, expensiveCompute(key));
  }
  return cache.get(key);
}
function expensiveCompute(k) { return `结果:${k}`; }
console.log(compute('a'), compute('a')); // 第二次直接命中缓存

// WeakRef(ES2021):更底层,持有"随时可能消失"的引用
// 用途:大缓存/对象池,允许 GC 在压力大时回收
let obj = { big: '数据' };
const ref = new WeakRef(obj);
console.log(ref.deref()?.big); // 数据,deref() 可能返回 undefined(已被回收)
obj = null;

```

⚠ 常见误区 5.2:用 Map 做对象缓存导致键永不释放

Map 对键是强引用: `cache.set(key, value)` 后,即使业务代码不再引用 `key`,Map 也拉着它。用 WeakMap 的代价是:不可遍历、不可知大小、键必须对象——所以"需要遍历的缓存"用 Map + 手动淘汰(定期删除),"只按对象查元数据"用 WeakMap。两者分工要清楚。

\$5.5 Buffer 与 ArrayBuffer:二进制数据

C 的 `unsigned char buf[1024]` 在 JS 中有两个对应物:**Buffer**(Node 专有,基于 Uint8Array,继承自标准 TypedArray,自带编码转换)与 **ArrayBuffer**(ECMAScript 标准,浏览器与 Node 通用,纯粹的字节容器)。生产规则:Node 环境优先 Buffer(API 齐全),跨端/Web 场景用 ArrayBuffer + DataView/TypedArray。

```

// Buffer:Node 二进制首选
const buf = Buffer.from('你好', 'utf8'); // 字符串 → 字节(UTF-8)
console.log(buf.length); // 6 ← 一个汉字 3 字节
console.log(buf); // <Buffer e4 bd a0 e5 a5 bd>
console.log(buf.toString('utf8')); // 你好,字节 → 字符串
console.log(buf.toString('hex')); // e4bda0e5a5bd,十六进制串

// 创建与读写
const b2 = Buffer.alloc(4); // 4 字节,清零(类似 calloc)
b2.writeUInt32BE(0x01020304, 0); // 大端写入
console.log(b2.toString('hex')); // 01020304
console.log(b2.readUInt16LE(2)); // 0x0304,小端读 16 位
b2[0] = 0xff; // 像 C 一样按下标改字节
console.log(b2[0]); // 255

// Buffer 与 C 字节数组对照:切片是视图不是拷贝
const big = Buffer.from([1, 2, 3, 4, 5, 6]);
const view = big.subarray(1, 4); // 视图,共享底层内存
view[0] = 99;
console.log(big); // <Buffer 01 63 03 04 05 06>

// ArrayBuffer:标准字节容器
const ab = new ArrayBuffer(8); // 8 字节
const u8 = new Uint8Array(ab); // 视图:8 个字节
u8[0] = 42;
console.log(u8[0]); // 42
const dv = new DataView(ab); // 任意类型读写
dv.setFloat64(0, 3.14);
console.log(dv.getFloat64(0)); // 3.14

// 对象 → JSON 字符串 → 字节(网络传输常用)
const payload = JSON.stringify({ ok: true, n: 7 });
const wire = Buffer.from(payload, 'utf8');
console.log(wire.toString('utf8')); // {"ok":true,"n":7}

```

⚠ 常见误区 5.3:Buffer.from(string) 的默认编码

默认是 utf8,但 Base64 场景常被搞错: Buffer.from(str, 'base64') 的 str 必须是合法 Base64;把普通字符串当 Base64 解码会得到乱码。另外 buffer.length 是字节数不是字符数,与字符串的 length 语义不同——文件大小、网络包大小都用字节数。

§5.6 资源释放惯用法:try/finally 与句柄关闭

C 用 free/fclose 释放;JS 的 GC 管内存,但 文件句柄、网络连接、定时器这些"外部资源"必须手动关闭。惯用法是 try/finally(第9章详解异常,这里先用结论):


```
const fs = require('node:fs');

// ❌ 危险:中途抛错就不关闭
function badWrite() {
  const f = fs.openSync('/tmp/out.txt', 'w');
  fs.writeSync(f, 'hello');
  // 若 writeSync 抛错,f 永远不关 → 句柄泄漏
  fs.closeSync(f);
}

// ✅ 正确:try/finally 保证释放
function goodWrite() {
  const f = fs.openSync('/tmp/out.txt', 'w');
  try {
    fs.writeSync(f, 'hello world');
  } finally {
    fs.closeSync(f);          // 无论是否抛错都执行
  }
}

goodWrite();
console.log('已安全写入并关闭');

// 文件句柄超限的表现:EMFILE: too many open files
// ulimit -n 调大 + 代码保证成对 open/close
```

§5.7 生产实践:泄漏检查清单与性能

🚀 生产泄漏检查清单(上线前过一遍)

- 全局容器(数组/Map)有没有无上限追加?加上限或定期清理
- 闭包有没有意外捕获大对象/大数组?把大对象放进块作用域
- setInterval/setTimeout 是否有清理路径?组件卸载、任务完成时 clear
- 事件监听是否在"不再需要"时 off 了?监听函数是否具名可移除?
- 缓存是 WeakMap 还是 Map?Map 是否带淘汰策略?
- 流(stream)有没有 drain/close 处理?HTTP 响应是否 end?
- 进程重启策略:崩溃自愈可以兜底"缓慢泄漏"(容器重启),但不能替代修复

```

// 性能注意:避免高频分配(与 C 的"频繁 malloc 慢"同理)
// ❌ 循环内创建字符串/数组
function buildBad(n) {
  let s = '';
  for (let i = 0; i < n; i++) {
    s += `item${i}`; // 每次 += 都创建新字符串
  }
  return s;
}

// ✅ 数组收集 + join(分配次数从 O(n) 降到 O(1) 次)
function buildGood(n) {
  const parts = new Array(n);
  for (let i = 0; i < n; i++) {
    parts[i] = `item${i}`;
  }
  return parts.join('');
}

console.log(buildGood(5)); // item0;item1;item2;item3;item4;

// 对象池:高频临时对象复用(游戏/高频消息场景)
class Pool {
  constructor(make) {
    this.make = make;
    this.free = [];
  }
  acquire() {
    return this.free.pop() ?? this.make();
  }
  release(obj) {
    this.free.push(obj);
  }
}

const pool = new Pool(() => ({ x: 0, y: 0 }));
const p = pool.acquire();
p.x = 1;
pool.release(p); // 归还复用,减少 GC 压力

```

工程建议 5.2:内存监控三件套

① 进程级: `process.memoryUsage()` 每 10 秒采样入库,画出 `heapUsed` 趋势线(持续上升 = 泄漏);② 引擎级:老生代增长率 `--max-old-space-size` 与 `--trace-gc` 观察 GC 频率,GC 越来越频繁且每次回收量趋近于零 = 泄漏;③ 应用级:压测工具(如 `autocannon`)在稳定 QPS 下跑 30 分钟,内存曲线应走平;持续爬升即泄漏,配合堆快照定位。

本章速查表

主题	速查要点
内存模型	对象都在堆;变量是引用;GC 回收不可达对象
回收算法	新生代 Scavenge(复制);老生代标记-清除 + 增量标记
GC 特性	自动、不可控、无 STW 长停顿(增量);--max-old-space-size 调上限
循环引用	不是泄漏!从根不可达即回收(与 C++ shared_ptr 不同)
泄漏一	全局容器无限增长 → 加上限/定期清理
泄漏二	闭包捕获大对象 → 只捕获所需变量
泄漏三	定时器未清理 → 明确停止条件 + clearInterval
泄漏四	监听器重复注册 → 具名函数 + off,或用 once
排查	--heap-prof / DevTools 堆快照 Delta / memoryUsage 趋势
WeakMap	键弱引用,不延长键生命周期;键必须对象,不可遍历
WeakRef	弱引用,deref() 可能 undefined;用于大缓存
Buffer	Node 二进制容器;length 是字节数;subarray 是视图
ArrayBuffer	标准字节容器 + TypedArray/DataView 视图
资源释放	句柄/连接/定时器用 try/finally 或清理函数成对释放
字符串拼接	循环内 += 慢;用数组 join 或模板字符串
对象池	高频临时对象复用,减少 GC 压力

最小必会清单

- 能解释"GC 回收不可达对象"并用两段代码演示引用断链
- 能说出新生代/老生代的回收算法名称与"增量标记"解决什么问题
- 能复现并修复四种泄漏场景中的至少三种
- 能说出 WeakMap 与 Map 的三个差异(键弱引用/键必须对象/不可遍历)
- 能说出 Buffer.length 与字符串 length 的差异,并完成 UTF-8 编解码
- 能写出 try/finally 保证文件句柄关闭的模板代码
- 能解释为什么 JS 没有悬垂指针与双重释放
- 能用 subarray 做出共享内存的切片视图
- 能说出循环拼接字符串的替代方案
- 能画出一个三层对象嵌套的引用图并指出断哪条边可回收子树

自测题

Q 1. 下面的代码会不会内存泄漏?为什么?`const a = {}; const b = {}; a.p = b; b.p = a; a = null; b = null;`

✅ 参考答案

不会。a 与 b 互相引用形成环,但环没有根可达,GC 的标记-清除从根出发,整个环不可达,会被回收。

Q 2. 说出四种内存泄漏场景,并各给出修复关键词。

✅ 参考答案

① 全局容器无限增长 → 上限/清理;② 闭包持有大对象 → 缩小捕获;③ 定时器未清理 → clearInterval;④ 监听器重复注册 → off/once。

Q 3. WeakMap 能遍历吗?它的键可以是字符串吗?用它做"用户会话缓存"合适吗?

✅ 参考答案

不能遍历,无 size;键必须是对象,字符串不行。用对象做键的会话元数据合适(键对象消失自动清理),需要遍历/统计的缓存用 Map。

Q 4. Buffer.from('你好','utf8').length 是多少?为什么?如何取回原字符串?

✅ 参考答案

6。'你好' 两个汉字,UTF-8 每字 3 字节,length 是字节数。toString('utf8') 还原。

Q 5. 简述 V8 为什么用分代回收,以及增量标记解决什么问题。

✅ 参考答案

大多数对象短命,分代让"小空间频繁复制"代替"全堆扫描",效率高。增量标记把单次大停顿拆成小片穿插执行,避免浏览器/服务卡顿(STW 时间缩短)。

Q 6. Node 里文件写入中途抛错会怎样?写出保证句柄关闭的模板。

✅ 参考答案

句柄泄漏,fd 不被释放,反复发生会 EMFILE。模板:openSync 后 try { 操作 } finally { closeSync }。

章末实验题:内存泄漏复现与修复实验室

实验 5:内存泄漏复现与修复实验室

实验目标:复现四种泄漏场景,观察堆增长,实现修复,并用 WeakMap/Buffer 完成收尾练习。

实验要求:

- 任务 1(覆盖:全局容器泄漏):模拟"全局日志数组无限增长",修复为定长环形缓冲
- 任务 2(覆盖:闭包泄漏):构造"闭包捕获大数组"案例,修复为只捕获所需字段
- 任务 3(覆盖:定时器泄漏):演示未清理的 setInterval 使进程不退出,修复为条件 clearInterval
- 任务 4(覆盖:事件监听泄漏):重复注册监听使 listenerCount 暴涨,修复为具名函数 + off/once
- 任务 5(覆盖:WeakMap/WeakRef):用 WeakMap 做对象元数据缓存,验证键对象置空后条目消失
- 任务 6(覆盖:Buffer/ArrayBuffer):把结构化数据编码为二进制并解码还原,演示字节长度与视图
- 任务 7(覆盖:try/finally 资源释放):实现文件写入 + 强制 finally 关闭,并打印 heapUsed 对比

实验环境:Node.js 20+,运行 `node leaklab.js`;建议同时运行 `node --expose-gc leaklab.js` 观察 GC 效果。

第一步 **写工具函数:**`memNow()` 返回 `heapUsed`;`leakScenario()` 循环创建对象并打点

第二步 **写四场景函数:**每个场景"先泄漏版(可选开关)→ 修复版",打点对比

第三步 **后写收尾:**WeakMap 生命周期演示 + Buffer 编解码 + finally 文件写入

✅ 参考答案要点

核心:每个场景用 "泄漏版/修复版" 双函数 + 内存打点输出对比,收尾覆盖 WeakMap/Buffer/finally。约 130 行。

```
// leaklab.js — 运行:node leaklab.js(建议加 --expose-gc 观察 GC)
'use strict';
const fs = require('node:fs');
const { EventEmitter } = require('node:events');

// 工具:内存快照(覆盖:process.memoryUsage、模板字符串)
function memNow() {
  return `${Math.round(process.memoryUsage().heapUsed / 1024)}KB`;
}

function gcHint() {
  if (typeof global.gc === 'function') {
    global.gc(); // --expose-gc 时强制回收(仅实验)
    return '(已手动 GC)';
  }
  return '(未启用 --expose-gc)';
}

// 场景一:全局容器泄漏 → 修复(覆盖:全局引用、数组上限)
const leakLog = [];
function leakLogEvent(e) { leakLog.push({ at: Date.now(), e }); }

const fixedLog = [];
function fixedLogEvent(e) {
  fixedLog.push({ at: Date.now(), e });
  if (fixedLog.length > 100) fixedLog.shift();
}

// 场景二:闭包泄漏 → 修复(覆盖:闭包捕获、块作用域)
function leakClosure() {
  const big = new Array(200000).fill('x');
  const id = 'u-1';
  return () => id;
}

function fixClosure() {
  const id = 'u-1';
  { const big = new Array(200000).fill('x'); console.log('big 仅块内可见:', big.length); }
  return () => id;
}

// 场景三:定时器泄漏 → 修复(覆盖:setInterval、clearInterval)
function leakTimer() {
  const data = [];
  const t = setInterval(() => data.push(Date.now()), 50);
  return () => { clearInterval(t); return data.length; };
}

function fixTimer() {
  const data = [];
  let t = setInterval(() => {
    if (data.length >= 5) {
      clearInterval(t); // 条件满足即停
    } else {
      data.push(Date.now());
    }
  }, 50);
  return () => data.length;
}
```

```

}

// 场景四:监听器泄漏 → 修复(覆盖:EventEmitter、on/off/once)
function runEmitterDemo() {
  const em = new EventEmitter();
  function handler(chunk) { if (chunk.length > 3) console.log('数据长度:', chunk.length); }
  for (let i = 0; i < 100; i++) em.on('data', handler); // 重复注册(演示)
  const before = em.listenerCount('data');
  em.off('data', handler); // 修复:全部移除
  em.once('data', (c) => console.log('once 消费:', c.length)); // 一次性
  em.emit('data', Buffer.from('abcd'));
  return { before, after: em.listenerCount('data') };
}

// 场景五:WeakMap 元数据(覆盖:WeakMap、set/get、置空)
function runWeakMapDemo() {
  const meta = new WeakMap();
  let user = { name: '李雷' };
  meta.set(user, { visits: 1 });
  user.visits = meta.get(user).visits;
  console.log('WeakMap 读取:', meta.get(user)?.visits);
  user = null;
  console.log('键置空后条目随 GC 消失(无法验证 size,WeakMap 不可遍历)');
}

// 场景六:Buffer 二进制(覆盖:Buffer、utf8/hex、subarray 视图)
function runBufferDemo() {
  const b = Buffer.from('你好JS', 'utf8');
  console.log('字节数:', b.length, '| hex:', b.toString('hex'));
  console.log('还原:', b.toString('utf8'));
  const big = Buffer.from([1, 2, 3, 4, 5]);
  const view = big.subarray(1, 3);
  view[0] = 99;
  console.log('视图改字节,底层同步变化:', big);
}

// 场景七:try/finally 文件句柄(覆盖:资源释放、finally)
function runFileDemo() {
  const fd = fs.openSync('/tmp/leaklab.txt', 'w');
  try {
    fs.writeSync(fd, 'hello leaklab\n');
    throw new Error('模拟中途出错');
  } catch (err) {
    console.log('捕获错误:', err.message);
  } finally {
    fs.closeSync(fd);
    console.log('句柄已在 finally 中关闭');
  }
}

// 主流程(覆盖:函数组织、对比输出)
function main() {
  console.log('== 场景一:全局日志 ==');
  for (let i = 0; i < 500; i++) { leakLogEvent(i); fixedLogEvent(i); }
  console.log('泄漏版长度:', leakLog.length, '| 修复版长度:', fixedLog.length);
}

```



```
console.log('== 场景二:闭包 ==');
const lc = leakClosure();
const fc = fixClosure();
console.log('泄漏版闭包返回值:', lc(), '| 修复版:', fc(), gcHint());

console.log('== 场景三:定时器 ==');
const leakTimerFn = leakTimer();
const fixTimerFn = fixTimer();
setTimeout(() => {
  console.log('泄漏版数据:', leakTimerFn(), '| 修复版数据:', fixTimerFn());
}, 300);

console.log('== 场景四:监听器 ==');
const { before, after } = runEmitterDemo();
console.log(`监听数 ${before} → ${after}`);

console.log('== 场景五:WeakMap ==');
runWeakMapDemo();

console.log('== 场景六:Buffer ==');
runBufferDemo();

console.log('== 场景七:资源释放 ==');
runFileDemo();

console.log('当前堆:', memNow());
}

main();
```

设计说明:四个泄漏场景各自"泄漏版/修复版"成对出现,输出对比数据让修复效果可见;场景三用 `setTimeout` 等待定时器跑完再统计,演示 `clearInterval` 的效果;场景四演示 `listenerCount` 从 100 归零;场景七用 `throw` 验证 `finally` 必然执行。建议先跑 `node leaklab.js`,再看 `node --expose-gc leaklab.js` 输出中的手动 GC 提示。

下一章预告:第6章 面向对象 —— 原型链、class 语法糖背后的 `prototype`、`#` 私有字段与 `getter/setter`,从 C 的 `struct`+函数指针进化到真正的面向对象。